



Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Sumário

- Plano de Otimização de Tokens — Fábrica Agêntica de Publicações 4
 - 1. Visão Geral 4
 - 2. Gargalos de Tokens — Ordem de Grandez 4
 - 2.1 Fase 2 — Manufatura de Capítulos (Maior consumidor) 4
 - 2.2 CLAUDE.md / AGENTS.md no Contexto 5
 - 2.3 Fase 2.5 — Peer Review (revisor-tecnico) 5
 - 2.4 Fase 1 — Pesquisa (pesquisador) 5
 - 2.5 Fase 0 — Esboço (/esbocar) 6
 - 2.6 Fase 3 — Compilação (compilador-abnt) 6
 - 3. Resumo de Ações Recomendadas 6
 - 4. Conclusão 7

Plano de Otimização de Tokens — Fábrica Agêntica de Publicações

Data: 2026-08-07 **Escopo:** Análise de gargalos de consumo de tokens no pipeline de produção

1. Visão Geral

A Fábrica Agêntica de Publicações utiliza LLMs em 4 fases do pipeline para produzir livros, TCCs, artigos científicos e e-books. Este documento identifica os maiores consumidores de tokens e propõe ações de redução mantendo a qualidade do output elevada.

2. Gargalos de Tokens — Ordem de Grandez

2.1 Fase 2 — Manufatura de Capítulos (Maior consumidor)

Causa: Cada capítulo dispara um `subagente-redator-capitulo` que executa: - RAG (`indexar-dossie.py`) — busca no dossiê - `estrategista` — decomposição em 3 pilares - `redator-eita` — escrita de 7 seções (EITA-V2) - Diagrama Mermaid + CI de código - Auto-validação

Impacto: Para um livro com 12 capítulos, são 12 subagentes rodando em lotes de 4. Cada um carrega o contexto completo da skill + dossiê + configuração. **~60-70% do custo total.**

Ações de redução: - Usar RAG mais agressivo (já parcialmente implementado via `indexar-dossie.py --topo 4`) — reduzir de 4 para 2-3 blocos por busca - Cache de contexto do dossiê: extrair um resumo de 500 tokens do dossiê e injetar no prompt do subagente em vez

de fazer RAG a cada capítulo - Lotes menores (2 em vez de 4) para reduzir pressão de contexto simultâneo, mas aumentar qualidade por lote

2.2 CLAUDE.md / AGENTS.md no Contexto

Causa: O `AGENTS.md` tem ~329 linhas e é carregado como system prompt em CADA sessão e CADA subagente. Somado ao `CLAUDE.md` (hardlink, mesmo conteúdo), são ~600+ linhas de instruções que entram no contexto automaticamente.

Impacto: ~15-20% do custo por sessão. Em 12 subagentes, são ~240% de overhead.

Ações de redução: - Comprimir AGENTS.md usando a skill `caveman-compress` — manter apenas regras ativas, remover exemplos históricos - Dividir em camadas: um `AGENTS.md` mínimo (regras + fluxo) + `AGENTS-REF.md` (referência completa, lido sob demanda) - Remover hardlinks duplicados — `.github/copilot-instructions.md`, `.windysurfrules`, `.clinerules` são cópias idênticas; apenas o `.cursor/rules/` é necessário para Cursor

2.3 Fase 2.5 — Peer Review (revisor-tecnico)

Causa: O `revisor-tecnico` lê todos os capítulos para auditar sobreposição, terminologia e truncamento. Para 12 capítulos de ~300 linhas cada, são ~3600 linhas de prosa no contexto.

Impacto: ~10-15% do custo total. Único, mas denso.

Ações de redução: - Auditoria incremental: revisar apenas capítulos novos ou alterados (usar git diff para identificar) - Resumo executivo: gerar um JSON com métricas (nº palavras, citações, blocos de código) em vez de ler a prosa inteira - Paralelizar em lotes: dividir capítulos em grupos de 3 e auditar em paralelo

2.4 Fase 1 — Pesquisa (pesquisador)

Causa: `WebSearch` + `WebFetch` para montar o dossiê. Cada busca retorna páginas inteiras que são processadas.

Impacto: ~5-10% do custo total. Variável conforme profundidade.

Ações de redução: - Limitar WebFetch a 2000 palavras por página (já implementado em alguns contextos) - Cache de pesquisas: salvar resultados de buscas anteriores para reaproveitar em livros derivados - Pesquisa focada: usar o `arquiteto` para definir EXATAMENTE o que pesquisar antes de pesquisar

2.5 Fase 0 — Esboço (/esbocar)

Causa: Interação inicial com o operador + planejamento da obra. Geralmente 1-3 turnos.

Impacto: ~2-3% do custo total. Mínimo.

Ações de redução: - Manter como está — é a fase mais curta e mais importante para qualidade

2.6 Fase 3 — Compilação (compilador-abnt)

Causa: Merge de capítulos + geração de pré/pós-textuais + compilação PDF. Majoritariamente determinístico (scripts Python), não LLM.

Impacto: ~1-2% do custo total. Quase zero tokens LLM.

Ações de redução: - Manter como está — já é otimizado via scripts determinísticos

3. Resumo de Ações Recomendadas

Prioridade	Ação	Redução Estimada	Esforço
1	Comprimir AGENTS.md (cave-man-compress)	−15-20% por sessão	Baixo
2	Cache de contexto do dossiê (resumo 500 tokens)	−10-15% na Fase 2	Médio
3	Auditoria incremental (só capítulos alterados)	−5-10% na Fase 2.5	Médio
4	RAG mais agressivo (topo 2 em vez de 4)	−3-5% na Fase 2	Baixo
5	Remover hardlinks duplicados de docs	−2-3% por sessão	Baixo

Redução total potencial: 35-50% dos tokens, mantendo qualidade.

4. Conclusão

O maior gargalo é a **Fase 2 (Manufatura de Capítulos)**, que consome 60-70% dos tokens devido à execução paralela de subagentes que carregam contexto completo. As ações de maior impacto e menor esforço são a compressão do AGENTS.md e a implementação de cache de contexto do dossiê — juntas, podem reduzir 25-35% do custo total sem perda de qualidade.

